

云计算运维培训课件：初识部署架构

目标：让完全 0 基础学员用一节课的时间，知道什么是“架构”、部署架构长什么样、以及 互联网架构为什么会从“单机”一步步演进到“云上分布式”。

场景：运维 / 云计算运维入门。

1. 这节课你会学到什么

学完你应该能回答

- “架构”到底是什么意思？和“服务器”“代码”“网络”有什么关系？
- 一个网站/系统上线，需要哪些组件？它们怎么连起来？
- 为什么不能一直用一台服务器？流量变大后会遇到哪些问题？
- 互联网部署架构常见的演进路线是什么？
- 运维在架构里主要负责什么？

关键词（先眼熟）

- 部署 / 上线、服务器、网络、带宽、域名、DNS、负载均衡、集群
 - 应用服务、数据库、缓存、对象存储、CDN
 - 高可用、可扩展、性能、成本、安全
-

2. 什么是“架构”？用一句话讲清楚

2.1 通俗定义

架构 = 为了实现某个业务目标，把系统需要的“组件”按一定规则组织起来，并规定它们如何连接与协作。

你可以把它理解为：

- 盖房子的“结构图”（哪里是承重墙、哪里走水电）

- 餐厅的“后厨流程”（点单→做菜→出餐→结账）
- 电脑的“硬件组合”（CPU、内存、硬盘怎么搭）

2.2 为什么运维要学架构？

因为运维负责让系统“活着、跑稳、跑快、扛得住”，而这些都取决于：

- 系统有哪些组件（服务器/数据库/缓存/网络……）
 - 部署在哪里（机房/云上/多地域）
 - 组件之间怎么连（网络、端口、权限、调用链路）
-

3. “部署架构”到底在描述什么？

3.1 部署架构回答的 3 个核心问题

1. 放哪儿：组件部署在什么机器/什么网络/什么区域？
2. 怎么连：用户怎么访问？服务之间怎么通信？
3. 怎么扩：流量大了怎么加机器？挂了怎么恢复？

3.2 你未来会经常看到的“架构图”元素

- 用户 / 浏览器 / App
 - DNS（把域名解析到 IP）
 - 入口层：Nginx / 负载均衡（LB）
 - 应用层：Web/业务服务（Java/PHP/Node...）
 - 数据层：MySQL/PostgreSQL、Redis
 - 存储层：对象存储（图片/视频）、块存储
 - 加速层：CDN
 - 运维保障：监控、日志、告警、备份、权限、安全组、防火墙
-

4. 从“一个网站”开始：最原始的单机部署

4.1 架构 0：一台服务器跑全部（最常见的起步）

用户 --> 域名DNS --> [一台服务器]

\- Web/应用

\- 数据库

\- 图片文件

4.2 优点

- 便宜、简单、部署快
- 适合学习、Demo、小业务

4.3 主要问题（流量一来就暴露）

- 单点故障：机器挂了=全站挂
- 性能瓶颈：CPU/内存/磁盘/网络任何一个满了都卡
- 扩展困难：想加机器？但数据库和文件怎么办？
- 维护风险：升级/重启都会影响全站

这一阶段运维最常做：装环境、部署应用、开端口、防火墙、备份数据库。

5. 架构为什么要演进？先理解“驱动力”

互联网架构演进通常是因为这几件事（记住：不是为了“高级”，而是被逼出来的）：

- 用户变多：访问量上升
- 业务变复杂：功能增多，代码更大
- 稳定性要求提高：不能动不动就宕机
- 数据变重要：数据库要安全、要备份、要容灾
- 成本压力：要用更合理的资源方式支撑增长

6. 架构 1：应用和数据库拆开（经典第一步）

6.1 部署形态

用户 --> DNS --> [应用服务器] --> [数据库服务器]

6.2 为什么要拆？

- 数据库更“金贵”，需要更强的磁盘、备份、权限隔离
- 应用经常改、经常发版；数据库不希望频繁被影响

6.3 运维关注点

- 网络连通（安全组/防火墙/端口）
 - 数据库备份与恢复演练
 - 账号权限最小化（应用账号只给需要的库表权限）
 - 监控：CPU、连接数、慢查询
-

7. 架构 2：加“负载均衡”，应用做成多台（能抗流量）

7.1 部署形态（应用集群）

```
用户 --> DNS --> [负载均衡LB] --> [应用1]
      --> [应用2]
      --> [应用3]
      |
      --> [数据库]
```

7.2 这一步解决什么？

- 扩容：应用不够就加机器（横向扩展）
- 高可用：一台应用挂了，LB 把流量导到其他机器

7.3 新问题出现了：会话与文件

- 会话（登录状态）：用户请求可能打到不同应用机
解决思路：共享会话（如 Redis）、或“会话保持”、或无状态化
- 上传文件：用户上传到应用1，访问时却到了应用2看不到
解决思路：共享存储 / 对象存储

8. 架构 3：引入缓存（Redis）与读写分离（更快更稳）

8.1 为什么要缓存？

很多请求是“读多写少”（例如商品详情、文章内容），如果每次都查数据库：

- 数据库压力大
- 响应慢
- 峰值容易崩

8.2 典型形态

用户 --> LB --> 应用集群 --> Redis缓存 --> 数据库
|
命中就返回
未命中查DB并回填

8.3 数据库读写分离（入门理解）

- 主库：负责写入（新增/修改）
- 从库：负责读取（查询）
- 好处：读能力变强
- 代价：有同步延迟、架构更复杂

9. 架构 4：静态资源上 CDN，对象存储承载文件

9.1 静态资源是什么？

图片、CSS、JS、视频、安装包等。

9.2 典型形态

用户 --> CDN(就近节点) --> 对象存储(源站)
用户 --> LB --> 应用集群 --> 数据库/缓存

9.3 好处

- 页面加载更快（就近访问）
- 应用服务器压力下降（不再扛大文件）

- 带宽成本更可控

10. 架构 5：微服务（了解即可，0 基础不深挖）

10.1 从“单体应用”到“多个服务”

当业务变大，代码很难在一个应用里维护，就会拆服务：

- 用户服务、订单服务、支付服务、库存服务……

10.2 形态示意

```
用户 --> 网关/API入口 --> 服务A  服务B  服务C
|         |         |
Redis  DB      MQ
```

这时运维关注更多：服务发现、配置中心、链路追踪、容器/K8s、发布策略等。

11. 互联网架构演进路线：一张“记忆地图”

按“最常见”演进顺序（不绝对，但很典型）：

1. 单机一把梭：应用+数据库都在一台
2. 拆数据库：应用和 DB 分离
3. 加负载均衡：应用多实例，变成集群
4. 加缓存：Redis 抗读、抗峰值
5. 数据库增强：主从、读写分离、分库分表（更后期）
6. 静态资源独立：对象存储 + CDN

- 7. 服务化/微服务：系统拆分，治理更复杂
 - 8. 多可用区/多地域容灾：更高等级可用性
-

12. 架构设计里最常见的 4 个目标（运维视角）

12.1 高可用（HA）

- 挂一台不影响整体
- 方式：冗余、健康检查、故障切换、主从、双活/多活（更高阶）

12.2 可扩展（Scalability）

- 流量上来能“加机器”
- 方式：水平扩容、无状态化、分层（入口/应用/数据）

12.3 高性能（Performance）

- 访问更快、响应更稳定
- 方式：缓存、CDN、数据库优化、连接池、异步化

12.4 安全与成本（Security & Cost）

- 安全：最小权限、隔离、审计、加密、WAF/防火墙
 - 成本：用合适规格、弹性伸缩、按需付费、冷热分层
-

13. 运维在“部署架构”中做什么？（入门版清单）

13.1 上线与交付

- 服务器/云资源准备（ECS、VPC、子网、安全组）

- 环境安装、发布部署、回滚预案
- 域名、证书（HTTPS）、反向代理（Nginx）

13.2 稳定性保障

- 监控：主机、应用、数据库、接口、业务指标
- 日志：采集、检索、留存策略
- 告警：阈值、降噪、值班与响应流程
- 备份：数据库备份、对象存储版本、恢复演练

13.3 性能与容量

- 压测与容量评估
 - 扩容方案：加应用实例、扩数据库、加缓存
 - 成本控制：规格、弹性、资源回收
-

14. 课堂练习：你来“读懂”一个最小互联网架构

14.1 题目

假设你要部署一个“图文博客网站”，需要：

- 用户访问网页（读）
- 用户登录、发文章（写）
- 文章有图片
- 访问量从每天 100 到每天 100 万

14.2 你应该能说出一个合理的演进

- 起步：单机（快）
- 流量上来：应用/数据库分离
- 再上来：LB + 多台应用

- 读多：Redis 缓存热点文章
 - 图片多：对象存储 + CDN
 - 进一步：数据库主从、读写分离
-

15. 课后总结（你必须记住的 6 句话）

1. 架构不是玄学，是组件如何组织与协作的方式。
 2. 部署架构关注“放哪儿、怎么连、怎么扩、怎么保命”。
 3. 单机简单但有单点故障，流量上来必然演进。
 4. 负载均衡 + 多应用实例是最典型的扩容方式。
 5. 缓存、CDN、对象存储是提升性能与降低压力的关键手段。
 6. 运维的核心目标：稳定、可用、可扩、可控、安全、成本合理。
-

16. 术语小词典（0 基础必备）

- DNS：把域名翻译成 IP 地址
 - 负载均衡（LB）：把访问流量分发到多台后端服务器
 - 集群：同一角色的多台机器一起提供服务
 - 高可用：部分组件故障不影响整体服务
 - 缓存（Redis）：把热点数据放在更快的存储里
 - CDN：把静态内容分发到离用户更近的节点
 - 对象存储：专门存放文件（图片/视频），可扩展、成本低
-

可选：讲师备注（建议授课节奏）

- 0-10 分钟：什么是架构、为什么要学
- 10-25 分钟：单机→拆 DB
- 25-40 分钟：LB→应用集群（核心）
- 40-55 分钟：缓存/对象存储/CDN
- 55-60 分钟：总结+练习互动

如果你希望我把它做成**可直接上课的 PPT 结构**（每页标题 + 讲稿 + 配图建议），告诉我你更偏好：

1. 60 分钟还是 90 分钟课程；
2. 你使用的云厂商（阿里云/腾讯云/华为云/AWS/通用）；
3. 想不想加一个“动手实验”环节（例如用 Nginx 做反向代理 + 简单负载均衡）。